



MARÉE

Creperia en Mazatlán, emprendimiento, expandirse

* Automatizar todo, llevarlo al sig nivel

Necesidades: App web que se adapte al teléfono

Royalty, lealtad, sistema para clientes frecuentes

Menú Digital

Portal administrativo * iPad

Noificaciones Whatsapp

App disponible en el teléfono

PPG

Empresa de pintas y retribuciones

* Sistema web de gestión de artículos y reservas

Lab 10/02/26

¿Por qué es necesario un sistema de control de versiones?

Un universo que nos permite controlar versiones por separado y juntarlas

git - version = Versión Github

git add = Agrega archivos que indiques uno por uno y con git add -A agrega todos los archivos y cambios

cd = Navegar a la carpeta donde cloné el repositorio

git status = Ayuda a conocer el status

git clone url = Clonar el repositorio

git commit -m "Crear HTML" En imperativo

Concact K = Limpieza terminal

git log = historial de cambios

ls = Lista de lo que hay

git push = Publicar los cambios

git branch -a = Todos los ramos

<!doctype html> = Creas documento html

```
<html>  
  <head>  
  </head>  
  <body>  
  </body>  
</html>
```

* Revisar Videos Programación básica HTML lo ocuparé para práctica 1

Lab 12/02

Para definir clase en CSS, escribe nombre clase

minecraft es para las id

Jerarquía en CSS Siempre precedencia lo más específico

1: Style

2: Id

3: Class

4: Selector general

== Compara valores

=== Compara valores y tipo de dato

Lalo 13/02

Framework = Marco de trabajo

Bootstrap es el más robusto de todos, se adapta a X

Materialize más rápido de aprender, no sabes cuando algo es digno de

Algo muy rápido de aprender

Tailwind son componentes básicos, costo por aprender

Grid, algo que no ves pero es la estructura de tablas o columnas, como cambia dependiendo la pantalla en la que se ve

Lalo 17/02

git checkout -b Ramo/interfaz = Se crea una nueva rama

Si ya existe es únicamente git checkout Ramo/interfaz

git merge Ramo/interfaz, Los cambios dentro de interfaz se agregan a la rama especificada

git pull siempre primero

Las cosas en git, nunca se van a perder, una explotación de versiones y puede manipular ambas

Lalo 24/02

JSON = Java Script Object Notation

Sirve para tener toda la configuración de proyecto, maneja todas las dependencias

Npm start = Default si agregas más Scripts poner npm run (Nombre Script)

Express funciona con middlewares

Para registrar nuevos uso USE con tres parámetros (Request, response, next)

y cada next, si que al siguiente middleware.

Cuando terminas es response.send para mandar respuesta al servidor

Labo 26/02

Const router permite pasar la info de un archivo a otro

modul.exports = router es para que otros archivos accedan a las funcionalidades de otro . pero decir dentro de la misma app: la

✓ para especificar la salida a archivos

Con / es la ruta absoluta Con status puedo mandar el error que yo quiera

Con .. Subes nivel en la carpeta

Hard Refresh: shift de click a refresh

EJS Se ejecuta en el servidor para crear html

— header = pedazo de código, no completo. Partials

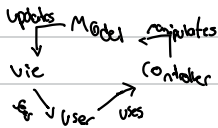
Para imprimir variable con EJS pones signo =

Labo 02/03

El código lo queremos en caps, para ver donde empieza y donde termina, no hacerlo así, genera una duda técnica.

Para evitar esto, aplicar SAP

El router recibe peticiones y redirige al controlador correspondiente



□ Pide a Controlador ⌚ Controlador ○ modelo H-O Interfaz

→ Petición Sincrónica

XAMPP

Después le da nombre a las columnas $PK = \text{Primary}$, $FK = \text{Index}$

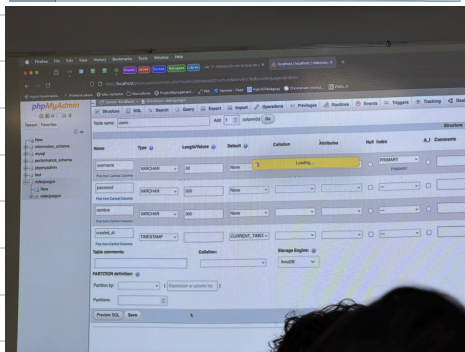
A_1 Auto-incrementer der eliert en la caselle

1mo DB Motor base de datos relacional

Solo un nombre de usuario y de base de datos

Catch como parametro recta funcion y si no se cumple la prueba, se ejecuta otra funcion (error)

Col Data = Metadatos, descripción de la columna



Lolo 23/03

AJAX

- Tecnología del lado del cliente (JS)
- Un medio de transporte (XML HTTP Request)
- Formato de cambio de datos (Originalmente XML)
- Tecnología de ejecución del lado del servidor (puede ser todo)

Lolo 06/04

¿Qué es un servicio web?

Un componente de software que se ejecuta en una computadora pero que puede ser consultado por varias aplicaciones o componentes que se ejecutan en otras computadoras o componentes.

¿Por qué usar servicios web?

- Promueven la portabilidad y reutilización

- Permiten transacciones B2B de una forma estandarizada y accesible

- Son independientes de plataforma y el lenguaje

- Permiten el desacoplamiento

- Pueden desarrollarse y mantenerse de manera independiente

- Permiten la descentralización y la redundancia

¿Qué tipos existen?

- SOAP (Simple Object Access Protocol): XML

- REST (Representational state transfer): Estilo arquitectónico de sistemas en Red, para transferir las informaciones entre los recursos como máquinas de estado.

Ricardo 10/02/26

Repaso Análisis de requerimientos. 6 tipos de requisitos

- **Requisito de tipo objetivo:** Es un enunciado (tesis) donde se expone a grandes rasgos una condición que debe de ser cumplida por el sistema. Al ser más específica describe la masa de un sistema.
- **Requisito de tipo informacional:** Describen los catálogos y campos y la relación que existe entre los campos. La información que se debe guardar, estructurar el flujo de la base de datos.
• Datos
• Catálogos
• Relaciones
- **Requisito de negocio:** Reglas, reglas o políticas del negocio que deben ser respetadas por el sistema.
- **Requisito de manera:** Define los ítemes que debe implementar el sistema para comunicarse con los usuarios y con otros sistemas.
• De control
• De comunicación
- **Requisitos funcionales:** Relación los ítemes que no a otros el sistema. • Los usuarios para alcanzar sus objetivos. • Se puede representar en todo o diagramas UML.
• Verbos en infinitivo Ejemplo: Abrir, registrar, imprimir
- **Requisitos NO funcionales:** Son ítemes que se le imponen al sistema a diseñar. • Se describen cuando todo (Atributos de calidad) tipo SMART.
• Usabilidad
• Rendimiento
• Disponibilidad
• Seguridad
• Mantenibilidad
• Escalabilidad
• Entre otros.

Identificar necesidad/problema

- **Requisito objetivo (TEXTO)**
- **Requisitos funcionales (Programa de casos de uso)**
- **Requisitos No funcionales (Atributos de calidad) preso**
- **Requisitos Informacional (Catálogos/Campos/Relaciones)**
- **Requisitos de Negocio preso (No negociables) Reglamentos**
- **Requisito Interfaz (Usuario/Comunicación)**

“Todo aquello que pudeser calculado, no debe ser almacenado”

Ricardo 12/02

En línea (Tiempo de respuesta inmediato, 2-3 seg)

• Diagrama de Gann

En lote (batch) (Sistema de jeringas pendientes por procesar) Ejemplo: Man

• Diagrama de Perdigón

Se acumula y se define regla para procesar UPS - No solo se va

Tiempo Real (Sistema de control aéreo, tiempo real)

* Si no hay contexto crearlo * Problema = Problema del negocio (Pensar como negocio pidiendo ganancias, etc.)

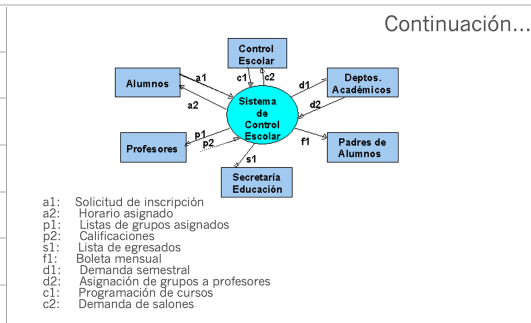
No Pensar únicamente como desarrollo de Software

Ricardo 16/02

Diagrama de contexto, realiza funcionalidades y datos, sirve para empezar la base de datos

Nomenclatura: Círculos, Flechas y rectángulos

Permite delimitar alcance del sistema, antes de al diseño de base de datos



Ricardo 17/02

Gestión de Alcance

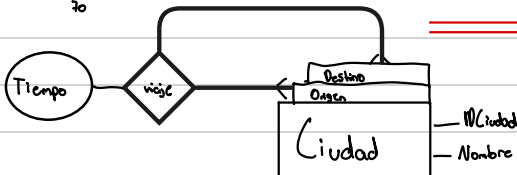
DL (Sistema ellipse, interesados rectángulo, Flujo de datos)

WBS = Work Breakdown Structure (Paquetes de trabajo, actividades y tareas)

Plan de trabajo (Tareas agrupadas, Tareas, esfuerzo planeado)

En el diagrama de contexto, llegar al nivel lógico y se agrega los datos que entran

Roles y especializaciones en entidad relación



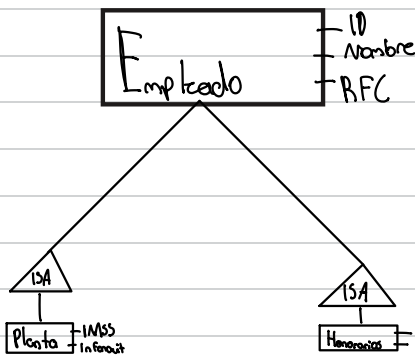
$\begin{cases} C1, Q10 \\ C2, C4 \\ C3, SJR \end{cases}$

$\uparrow$

$\begin{matrix} \text{Ciudad} & (\text{PK } \text{IDCiudad}, \text{Nombre}) \\ \text{Viajes} & (\text{IDc origen}, \text{PK } \text{IDc destino}, \text{Tiempo}) \end{matrix}$

$\text{PK} \downarrow$

$\begin{cases} C1, C3, 50 \\ C2, C3, 40 \\ C2, C1, 60 \\ C3, C1, 70 \end{cases}$



* Preguntas claras o Borrado

* Entrevistas para ir validando y refinando ideas

* WBS se irá expandiendo, primer enfoque en Ovale 2, y el resto casos del 3

Caso 1 Farm



* Rechar WBS

* Reglas de negocio

* Labor clínicas y lecturas

Ricardo 19/02

* A lo largo del tiempo para el MER, pasar en ese momento de diseñar la base de datos.

* Modificar relación de Equipo y presidente

Poner en un ROL, Local y visitante

Entidad posición, en el tiempo no siempre es la misma, relacionar con partido

Estadio, cambiar un equipo puede jugar de local en varios estadios

Patrón de diseño = Soluciones o guías

Diagrama de Clases tipo estructura Aquí si pensar en base de datos

Base de datos relacional = No métodos

Tres Relaciones

- Simple
- Agregación
- Composición
- Herencia

! Nombre de la clase = objeto

Cardinalidad = MER

Multiplicidad = DC

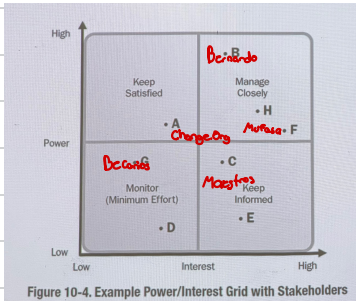
* Siempre ir de lo más simple a lo más complejo

Polimorfismo, cuando dos clases heredan lo mismo pero de manera distinta

Pensar que es lo más importante para presentar en el video o en clase

- Descripción de actores

Para identificar interesados utilizar



• Borrando

• Mufasa

• Change.org

• Maestros

• Beccarios

Laboratorio

- Creación del diagrama de contexto
- Elaborar el MER (Análisis)
- Traducir MER a MR (Diseño)
- Construir la BD (Usar GUI, código)
- Cargar tablas (Manual, importado archivos, Script)
- Generar Consultas (GUI, SQL)

Unidad = Llaves primarias

Integridad Referencial = Llaves foráneas

Picardo 26/02

Gestión de Comunicación

Proceso para construir una DB Relacional

- 6 pasos
 - Creación del diagrama de contexto
 - Elaborar el MER (Análisis)
 - Traducir MER a MR (Diseño)
 - Construir la BD (Usar GUI, código)
 - Cargar tablas (Manual, importado archivos, Script)
 - Generar Consultas (GUI, SQL)

MS Acces

Modelo Relacional - Álgebra relacional

Modelo Conceptual MER, hay dos Conceptos

Grado y Cardinalidad

Grado: Es el número de entidades que participan en una relación

Cardinalidad: Es el número de interacciones entre entidades (dúplas) en el tiempo

Modelo lógico MR

Grado: Número de campos en una relación

Cardinalidad: Número de dúplas en una tabla

SQL es interpretado, no compilado

Prejoín, intjoín, postjoín

Unión = $\cup$ Operador binario, conmutativo. Operandos tienen que tener mismo grado y mismo dominio, resultado: aquellas tuplas que existen en 1 y 2

Intersección = $\cap$ Operador binario, conmutativo. Operandos tienen que tener mismo grado y dominio. Aquellas tuplas que existen en 1 y existen en 2

Diferencia = $-$ Operador binario, no conmutativo. Operandos tienen que tener mismo grado y mismo dominio. Tuplas que están en a pero no en b

Proyección = $\pi_{\text{parámetro1}, \text{parámetro2}}()$ Operador Unario, no conmutativo, donde operando no tiene restricción de grado y dominio

Selección = $\sigma_{\text{condición}}()$ Operador Unario, que modifica la cardinalidad y especificar condición a cumplir, no afecta al grado

Join Natural $\Rightarrow \bowtie$ Operador binario, conmutativo, no son del mismo grado ni dominio y tiene que tener una columna en común

Theta Join = $\theta_{\text{condición}}$ Operador binario, conmutativo, grado y dominio no importa, no hay columna en común, Condición sobre los valores que tienen los campos

Producto Cartesiano: Todas las combinaciones que se pueden hacer entre conjuntos $\times$ binario, regresan todas las combinaciones entre registros N^2

Cliente (Pk)

Facturas (PK ^{AK} NO Fac, ^{AK} Stakes, ^{AK} ficher)

Verbo = Relación

Productos (PK, Cítricos, ^{Ak} Nombre, ^{Ak} descripción, ^{Ak} precio, ^{Ak} estado)

Sustentivo = Entidad

Proveedor (PK RFCP, Razón Social, Domicilio, Contacto)

Venta (Continued)

Surte (FK Fecha, Cantidad)

Clave principal: Identificar principal por una tabla, asig. en unid. ad.

Llave foránea: Llave que enlaza a los datos de dos tablas, asegura integridad referencial

Usar alterne: Cendenta a PK quegen los dolos actuando como identificador Secundario

Consensus

② Π description (5 "denominación" "generalidades" (Papeles) < (Materiales) < (Entregas))

① π Description ($\sigma_{\text{clone}} > 2000$ and < 100 (Molecules))

③ π (Proceduras) $\times$ (Entregas))

④ $P_1 = \pi_{APC}(O_{\text{Bioscience}}) = \text{MENA}^2 \text{ and } P_{\text{Bioscience}} = 1/4 (\text{Materials} \times \text{Energy} \times \text{Proteostasis})$

$$B_2 = \pi_{RFC}(\sigma_{\text{Retroceder}} = \text{"SAN LUIS"} \text{ and } \sigma_{\text{Retroceder}} = \text{"Leina"} \wedge \frac{1}{4} (\text{Materialien} < \text{Entregon} \times \text{Proyecto}))$$
$$R_1 \cap R_2$$

⑤ π permutation, disposition, rearrangement (Order $> 01/01/1993$ and Order $< 01/01/1993$ (Proyecto $\times$ Paralelos $\times$ Enteros $\times$ Matrices)))

Consultas 2

① $\tau_{\text{Titolo}}(\sigma_{\text{Nome} = \text{"Stroon Stone"}}(\text{Elenco}))$

② Γ Nombre, importantes (0 = "Tor (Luis)" (Padrón) < (Petite) < (Elenco)

(3)

④ π Nombre (σ_{220} = 'Fenerino' (Actor) < Reporto

⑤ IT Piracy, Telefilm, Sex & non-studio "Episodes" and non-studio

$R_2 = \pi$ Piacini, Telefono, Sacco (0)

Ricardo 03/03

Algebra relacional a SQL

Select*
FROM Materiales = Mostrar la tabla

Select Descripción
From Materiales as M, Entregas as E = $\pi_{(Descripción/Materiales > Entregas)}$
Where M.Clave = E.Clave

Having = Para funciones agregadas

Select Descripción
From Materiales as M INNER JOIN Entregas as E
ON M.Clave = E.Clave

Select M.Clave, Descripción, Fecha
From Materiales as M INNER JOIN Entregas as E = Para mostrar tablas de Clave, descripción y Fecha
ON M.Clave = E.Clave

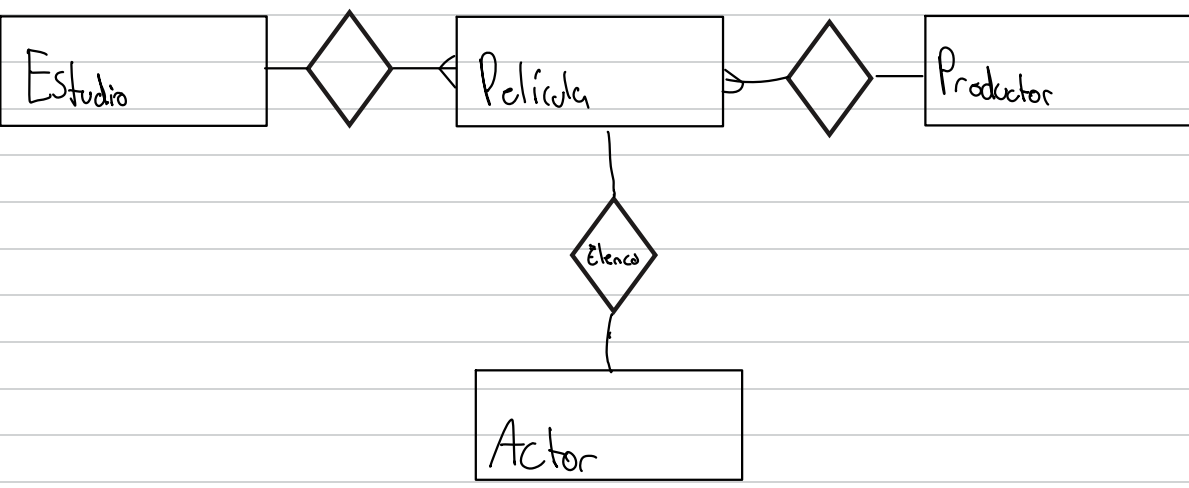
Select Descripción, Count(*) as "Número de entregas" Descripción del material y el total de entregas
From Materiales as M, Entregas as E
Where M.Clave = E.Clave
Group by Descripción

Select Descripción, SUM(Cantidad) as "Total de unidades" Descripción del material y el total de unidades entregadas
From Materiales as M, Entregas as E
Where M.Clave = E.Clave
Group by Descripción
Having Sum(Cantidad) > 1000
Order by Sum(Cantidad) Desc

(Select Description
From Materiales)

(Select Description
From Materiales M, Entregas E
Where M.Clave=E.Clave)

ACT Clase



$\Pi_{Nombre, Importantes} (\sigma_{Nombre = "Tom Cruise" (Productor \times (Pelicula \times Elenco))})$

$\Pi_{Nombre, Telefono} (\sigma_{p.Nombre = "George Lucas" (Actor \times Elenco \times (Pelicula \times Productor))})$

$\Pi_{Nombre} (\sigma_{fechaemision > 1985} (Actor \times Elenco \times (Pelicula)))$

Ricardo 05/03

Resumen

Consultas con funciones agregadas

- Count(*), Sum(), Avg(), Min(), Max(), STD()

Select

From

Where

Group by

Having

Order by

-- DDL Data definition Language Modifica Estructuras

Create, Alter, Drop

-- DML Modifica Contenido

Insert, Delete, Update,

Ejemplo

CREATE TABLE Materiales

(Clave Int NOT NULL,

 Descripcion varchar(100),

 Precio Numeric(6,2)

)

ALTER TABLE Materiales ADD CONSTRAINT pkMateriales Primary Key (Clave)

INSERT INTO Materiales VALUES (1000, 'Martillo mano acero', 120)

UPDATE Materiales SET Precio = Precio * 1.10 WHERE Clave = 1000

DELETE Materiales WHERE Clave = 1000

Picudo 09/03

Creación de BD y carga de tablas

Uso de Scripts

Garantizar unicidad e integridad referencial (códigos)

DDL, DML

Select, From, Where, Group by, Having, Order by Having condiciones sobre funciones agregadas

Diseño de Pruebas

• Creación diagrama de caso de uso

• Detallar caso de uso (Plantilla o diagrama de actividades)

• Elaborar un árbol de decisiones

• Elaborar tabla con los diferentes escenarios

• Elaborar tabla con los nombres de cada escenario

• Elaborar tabla con el tipo de capacidad/módulo/ANM de cada escenario

• Elaborar tabla con datos de entrada reales/representativos

• Agregar 2 campos nuevos, tener evidencia histórica

• Resultado obtenido
• Respuesta esperada

Consultas LAB

① Select Nombre, SUM(Sueldo) as 'Ingresos Totales'

FROM Elenco

GROUP BY Nombre

ORDER BY SUM(Sueldo) DESC

②

Select from elenco SUM(Sueldo) as 'Monto Total'

From Pelicula

Where año between 1980 and 1989

Group by pelicula

Order by SUM(Sueldo) Desc;

③ Select Nombre, AVG(Sueldo) as 'Sueldo Promedio'

From Elenco

Group by 'Sueldo Promedio' > 3000000

Having Sexo = 'Hombres'

Order by AVG(Sueldo) Desc;

④ Select titulo, año, MAX(Sueldo) as 'Presupuesto en de películas'

From Pelicula

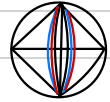
Group by Titulo

Order by MAX(Sueldo) Desc

⑤ Select nombre, Sueldo As 'Actriz mejor pagada'

From elenco, Actor

Where elenco.nombre = Actor.nombre AND Actor.Sexo = 'femenino'



```
SELECT nombre, AVG(sueldo) as 'Promedio de Sueldos'
FROM Elenco, Actor
WHERE Elenco.nombre = Actor.nombre AND Actor.sexo = 'male'
Group by Actor.nombre
HAVING AVG(sueldo) > 5,000,000
order by AVG(sueldo) DESC
```

```
SELECT MAX(sueldo) as "Max pagada"
FROM Elenco, Actor
WHERE Elenco.nombre = Actor.nombre AND Actor.sexo = "female"
```

Ricardo 06/04

Una transacción es una secuencia de operaciones. Conocidas como propiedades ACID. (Atomicidad, Coherencia, aislamiento y durabilidad)

Atomicidad: Una transacción debe ser una unidad atómica de trabajo, tanto si se realizan todas sus modificaciones en los datos, como si no se realiza ninguna de ellas.

Coherencia: Cuando finaliza, una transacción debe dejar todos los datos en un estado coherente. En una base de datos relacional, se deben aplicar todas las reglas a las modificaciones de la transacción para mantener la integridad de todos los datos. Todas las estructuras internas de datos, como índices de árbol B o listas doblemente vinculadas, deben estar correctas al final de la transacción.

Aislamiento: Las modificaciones realizadas por transacciones simultáneas se deben aislar de las modificaciones llevadas a cabo por otras transacciones simultáneas. Una transacción ve los datos en el estado en que estaban antes de que otra transacción simultánea los modificara o después de que la segunda transacción se haya concluido, pero no ve un estado intermedio. Esto se conoce como serializabilidad debido a que su resultado es la capacidad de volver a cargar los datos iniciales y reproducir una serie de transacciones para finalizar con los datos en el mismo estado en que estaban después de realizar las transacciones originales.

La serialización es un estado de base de datos que se obtiene mediante la ejecución de un conjunto de transacciones simultáneas y es equivalente al estado que se obtendría si se ejecutara el conjunto de transacciones en serie y por orden.

Durabilidad: Una vez concluida una transacción, sus efectos son permanentes en el sistema. Las modificaciones persisten aún en el caso de producirse un error del sistema.

Normalización

Tercer 6 Formas. Primera Segunda tercera, forma normal de Boye -Codd, Cuarta Quinta

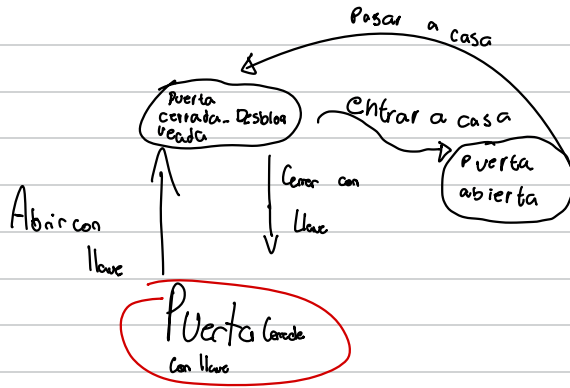
Diagramas de estado

Definición: Máquina de estados

- > Una máquina de estados es un comportamiento que especifica las secuencias de estados por las que pasa un objeto a lo largo de su vida en respuesta a eventos, junto con sus respuestas a esos eventos (*Booch, Rumbaugh, Jacobson*)

Introducción diagramas de estado

- > Los diagramas de estados son una técnica conocida para describir el comportamiento de un sistema. En situaciones particulares nos ayudan a clarificar casos de uso.
- > Describen todos los estados posibles en los que puede entrar un objeto particular y la manera en que cambia el estado del objeto, como resultado de los eventos que llegan a él.
- > Generalmente se dibujan para “una sola clase”, mostrando el comportamiento de un solo objeto durante todo su ciclo de vida.



Picando

Pruebas Heurísticas

Para hacer las es necesario tener: Prototipo / Sistema Alpha

Pasos:

- Conocer los 10 Reglas
 - Conocer el formato a llenar
- 1- HE Good / Problem
 - 2- Descripción
 - 3- Evidencia
 - 4- Explicación
 - 5- Severidad

3- Identificar incidentes llenando reportes (El equipo de trabajo)

4- El líder de pruebas es responsable de analizar los reportes y priorizarlos

5- El líder asigna reportes a ser atendidos a cada integrante del equipo

6- El líder valida la atención a los reportes

Socio formador

Ahorar tiempo en el proceso de Review, actualmente se toman 2 horas por ingeniero solamente para recibir información

Talleres

- Como sistema puede recibir info de stack
- Como comunicarse con chat gpt
- Como correr el proyecto en un servidor y despliegue de forma correcta

• La información no debe estar expuesta al público

Control de acceso basado en roles para crear usuario

• Solo los usuarios pueden ver lo raro (Trabajadores de Change.org)

- Admin

- Manager

- Usuario/Emppleado

- Product Manager

Sistema de autenticación básico

- No contraseñas con tanto plazo
- Nunca meter las llaves directamente en el código

* Plantillas de aceptación de registros

Entidades

Equipos → Empleados

Tareas →

Logros → Proyectos

Metas → Proyecto

Recordando 20/10/2

Uso en buen control de versiones, Siempre antes de empezar cambios a más nivel en equipo

- Pedir por las cosas por requisitos
- No seguir tareas o esta para dar actividad

Dividirlo por partes

Código Producción = Código Para producción

Conventional commits, como estándar de redactor commits, los msg y el commit si tienen que ir así.

No confundir actividades con funcionalidades

• Programa WBS

• Programa de actividad - MER

• Guía de certificación

• Programa de casos de uso

• Mapa de sitio?